

Oscp+ — compiled path

Compiled from `/home/dnikolic/Documents/Learning/Plan ucenja/paths/Oscp+`. Canonical sources stay in place.

Phase — 01-pen200-path-framing

Directory: `01-pen200-path-framing/`

Source: `01-pen200-path-framing/01-program-overview-and-pedagogy.md`

Oscp+ / PEN-200 — program overview and learning model

This trace is for building a solid base in **Linux**, **networks**, **web security**, **Windows enterprise**, **Active Directory**, and practical **systems analysis**. The focus is not only on tools, but on **how systems behave**, how **misconfiguration mistakes** typically appear, and how issues can be **recognized**, **reproduced**, and **remediated** responsibly.

Topics are organized in **phases** as numbered `paths/Oscp+/<NN>-area/` trees: Linux foundations → Networking → introductory cyber foundations → Web/backend/auth → systematic Burp tooling → Windows / AD enterprise context → **Kali-based toolchain familiarity** → **deep enumeration methodology** → **guided lab ladders toward PEN-200**. Earlier blocks deliberately feed later ones.

Stress is on **hands-on work**:

Practice surface	Examples
Local labs	VMs, scripted exercises
Network analysis	Captures and logs you own authorization to inspect
Reproduction	Controlled repro of misconfigurations and failures
Documentation	Structured notes and findings

The outcome is **not** memorizing individual tools alone, but cultivating **technical thinking**, repeatable **methods**, and the ability to **analyze** complex systems safely.

This is **long-horizon** study — steady daily practice beats rushing material. Aim for depth of understanding, not completion speed alone.

Source: `01-pen200-path-framing/02-phase-one-primary-resources-and-environment.md`

Phase one — Linux / CLI foundations (resources & environment)

Phase one ends when you work in a Linux shell **without hesitation**, grasp a **server-oriented** mental model, and perform **basic administration** comfortably.

Core goal (not “memorize Linux”)

You want a **low-friction CLI**, working knowledge of the **filesystem**, **users / groups / permissions**, **processes**, **packages**, **logging**, **systemd**, **SSH**, **text tooling**, **Bash basics**, and a **Linux server mindset**.

Primary resources

LabEx (Linux Journey-style Linux learning)

Site: <https://labex.io/>

Complete these tracks end-to-end:

- Grasshopper
- Text-Fu
- User Management
- Process Utilization
- Package Management
- Filesystem
- Networking Nomad
- Init / Systemd
- Logging

Extra emphasis: Grasshopper, Text-Fu, Networking Nomad.

Udemy — Linux Administration Bootcamp

Course: [Linux Administration Bootcamp](#)

Focus only on:

- Linux fundamentals
- Filesystem
- Permissions
- Users
- Process management
- Package management
- Networking sections
- Logging
- systemd
- SSH
- cron

Defer for now: LDAP, Kerberos, deep LVM, heavy enterprise storage topics.

Practice environment (required)

- **Ubuntu VM** (or equivalent you control)
- **Terminal every day**
- A **local practice folder**, e.g.:

```
mkdir training
cd training
```

Phase — 02-linux-foundations-phase

Directory: `02-linux-foundations-phase/`

Source: `02-linux-foundations-phase/01-terminal-and-filesystem-foundations.md`

Unit 01 — Terminal and filesystem foundations

Theme

Shell basics and filesystem navigation.

LabEx (Linux Journey)

Finish:

- Getting Started
- Linux History
- Choosing Distribution
- Ubuntu
- Debian
- The Shell
- Command Line

Udemy — Linux Administration Bootcamp

Sections:

- Linux intro
- Terminal basics

Commands to practice

`pwd, ls, cd, mkdir, touch, cp, mv, rm, clear, history`

Exercise

```
mkdir demo
cd demo
touch oauth.txt
cp oauth.txt oauth2.txt
mv oauth2.txt backup.txt
ls
pwd
```

Topic checklist

- Shell and terminal
- Filesystem
- Directories and files
- Relative vs absolute paths
- Command history

Learning outcome

You can create, copy, move, and remove files and directories and explain where you are in the tree.

Source: `02-linux-foundations-phase/02-files-and-text-processing.md`

Unit 02 — Files and text processing

Theme

Reading and shaping text on the command line.

LabEx

Start / continue:

- **Text-Fu**

Udemy — Linux Administration Bootcamp

Sections:

- Filesystem
- File operations

Commands to practice

`cat`, `less`, `head`, `tail`, `wc`, `sort`, `uniq`, `cut`

Exercise

```
echo oauth > log.txt
echo jwt >> log.txt
cat log.txt
sort log.txt
wc log.txt
```

Topic checklist

- stdin / stdout / stderr
- Text processing
- File manipulation

Learning outcome

You can inspect file contents, count lines/words, and sort/dedupe simple text without opening an editor for every step.

Source: `02-linux-foundations-phase/03-grep-pipes-and-bash-basics.md`

Unit 03 — grep, pipelines, and Bash basics

Theme

Finding data and composing small shell programs.

LabEx

Finish:

- **Text-Fu**

Add basic Bash:

- Variables
- `if`
- `for` / `while`
- Exit codes

Commands to practice

`grep`, `find`, `locate`, `tee`, `xargs`, `awk '{print $1}'`, `sed 's/old/new/g'`

Pipelines

Practice composing with `|`.

Example:

```
cat app.log | grep oauth
```

Bash basics

Example script pattern:

```
#!/bin/bash
for f in *.log; do
    grep ERROR "$f"
done
```

Exercise

```
find . -name "*.php"
find . -name "*.yaml"
grep oauth app.log
cat app.log | awk '{print $1}'
sed 's/oauth/oauth2/g' log.txt
```

Topic checklist

- `grep`, `find`, `locate`
- Pipes, `tee`, `xargs`
- Bash variables and loops
- Exit codes

Learning outcome

You can search a tree, filter logs, and write short loops without copy-pasting fragile one-liners you do not understand.

Source: 02-linux-foundations-phase/04-users-groups-and-permissions.md

Unit 04 — Users, groups, and permissions

Theme

Who can access what on the system.

LabEx

Finish:

- User Management
- Permissions

Udemy — Linux Administration Bootcamp

Sections:

- Users
- Permissions

Commands to practice

`chmod`, `chown`, `sudo`, `groups`

Exercise

```
touch secret.txt
chmod 600 secret.txt
chmod 755 script.sh
```

Interpret a mode string such as `-rwxr-xr-x` in your own words (owner / group / others; read/write/execute).

Topic checklist

- Users and groups
- `sudo`
- POSIX permissions (`rwx`)
- `chmod` / `chown`

Learning outcome

You can set sane defaults for scripts and sensitive files and explain why a permission change fixes or breaks access.

Source: 02-linux-foundations-phase/05-processes-and-services.md

Unit 05 — Processes and services

Theme

What runs on the system and how it is supervised.

LabEx

Finish:

- **Process Utilization**

Udemy — Linux Administration Bootcamp

Sections:

- Process management

Commands to practice

```
ps, top, kill, jobs, systemctl
```

Exercise

```
ps aux
top
kill <PID>
systemctl status ssh
```

Topic checklist

- Processes and PIDs
- Foreground / background jobs
- Services and **systemd** unit interaction at a basic level

Learning outcome

You can list processes, identify a runaway job, stop it safely, and check whether a service is active.

Source: `02-linux-foundations-phase/06-packages-logs-and-systemd.md`

Unit 06 — Packages, logs, and systemd

Theme

Keeping the system maintained and observable.

LabEx

Finish:

- Package Management
- Init / Systemd
- Logging

Udemy — Linux Administration Bootcamp

Sections:

- `apt` (or distribution equivalent)
- Logs
- `journalctl`

Commands to practice

`apt`, `dpkg`, `journalctl`, `tail`, `tar -czf backup.tar.gz demo/`, `tar -xzf backup.tar.gz`

Exercise

```
journalctl -xe  
tail -100 /var/log/syslog
```

(Paths may differ by distro; adapt if you are not on Debian/Ubuntu.)

Topic checklist

- Package management (`apt` / `dpkg` class)
- Logs: `journalctl`, classic syslog files
- systemd concepts tied to **enable/start/status**
- Archives: `tar` + compression

Learning outcome

You can install/update packages, inspect recent system events, and create/restore a simple tarball backup of a practice directory.

Source: `02-linux-foundations-phase/07-ssh-curl-and-networking-basics.md`

Unit 07 — SSH, curl, and Linux networking basics

Theme

Talking to other hosts like a server operator.

Udemy — Linux Administration Bootcamp

Sections:

- SSH
- Networking introduction

Commands to practice

`ssh`, `scp`, `curl`, `wget`, `ssh-keygen`, reading `~/.ssh/id_rsa.pub`, `tmux` install and basic use

Exercise

```
curl https://google.com  
curl -I https://google.com
```

Set up **SSH between VMs you control**: generate a key pair, install the public key, connect without password where appropriate.

Practice **tmux**: prefix **Ctrl+B**, detach/reattach, split panes basics.

Topic checklist

- SSH client usage and keys
- `scp` for file transfer

- `curl` /headers at a pragmatic level

Learning outcome

You can connect to your lab machines reliably, inspect HTTP responses at the CLI, and keep long sessions organized with `tmux`.

Source: `02-linux-foundations-phase/08-integration-drills-and-completion-criteria.md`

Unit 08 — Integration drills and Phase one completion criteria

Theme

Closed-book style rehearsal (no cheating with search during the drill block — then verify gaps afterward).

Scenarios (do without looking up)

1. Locate a relevant log file for a failing service scenario you invent on your VM.
2. Filter that log with `grep`.
3. Change permissions on a file to match two different intent cases (secret vs runnable script).
4. Restart or reload a practice service with `systemctl`.
5. Find a process by pattern; terminate it cleanly.
6. Install a small package you have not installed before (then remove if you wish).

Toolkit you should actively use:

`find`, `grep`, `chmod`, `systemctl`, `journalctl`, `kill`, `apt`

Phase one exit bar (skills)

Commands you should handle **without relying on external search mid-session** for basic flags:

`grep`, `find`, `chmod`, `chown`, `ps`, `kill`, `systemctl`, `journalctl`, `curl`, `ssh`, `scp`, `tail`, `less`, `cat`, `apt`, `sudo`

Concepts you must be able to explain briefly:

Area	Checkpoint
Filesystem	Where things live; absolute vs relative paths
Permissions	Users, groups, <code>rx</code> , when <code>sudo</code> is appropriate
Processes	PID, stopping processes, basics of service supervision
Logs	Finding recent errors; journal vs files
Text	Pipes; simple <code>grep</code> / <code>awk</code> / <code>sed</code> workflows
Bash	Variables, loops, exit codes

Stretch (optional sanity)

Rebuild one exercise from Units 01-07 from memory once per topic.

Deliverable

Write a half-page note: **three weaknesses** surfaced during Unit 08 and **one deliberate repetition** you plan explicitly (no implied calendar).

Phase — 03-networking-foundations

Directory: [03-networking-foundations/](#)

Source: [03-networking-foundations/01-primary-resources-practice-setup.md](#)

Phase two — Networking foundations (goals, resources, lab setup)

Phase two ends when you can explain **packet path, why a firewall blocked something, DNS, NAT, subnets, routing**, basic **traffic capture**, and everyday **network debugging** — without treating networking as unexplained magic.

Exit mindset

Avoid	Aim for
"Networking is magic"	"I see the system and the flow."

You should articulate at least conceptually:

- How traffic moves **browser → resolver → routing → TCP → TLS → HTTP → server**.
- Typical failure classes: **wrong DNS, no route, blocked port, NAT hairpin, MTU fragmentation**, stale **ARP/neighbor cache**.

Primary resources — Practical Networking

Site: <https://www.practicalnetworking.net/>

Work through:

- OSI model
- TCP vs UDP
- IP addressing
- Subnetting & CIDR
- DNS
- DHCP
- NAT
- ARP
- Routing
- Packet flow

This is your **anchor** syllabus.

Primary resources — Udemy (selected only)

Course: [Networking Fundamentals — CCNA start](#)

If the slug changes, locate the enrollment page you purchased and substitute the authoritative URL yourself.

Topics to cover:

- OSI vs TCP/IP practical framing
- IP addressing
- Subnetting & CIDR
- DNS
- DHCP
- NAT
- Routing fundamentals

Skip for this phase: Deep Cisco IOS lab grind, switching/VLAN internals at enterprise depth. **Outcome target is OSCP-aligned foundation**, not a CCNA trophy.

Supplemental — NetworkChuck (spot use only)

Channel: <https://www.youtube.com/@NetworkChuck>

Search when a topic does not “click”:

- “NetworkChuck OSI model”
- “NetworkChuck TCP UDP”
- “NetworkChuck subnetting”

Not a substitute for Practical Networking sequence.

TryHackMe

Platform: <https://tryhackme.com/>

Rooms / paths:

1. Locate the **Wireshark**-style room and practise **captures ethically** — only hosts you control or THM-hosted targets.
2. **Network Fundamentals** path modules.

Later (after competence here): optional **Pre Security** warmup path if you rotate into Phase three materials.

Subnet rehearsal (muscle memory)

Site: <https://subnettingpractice.com/>

Use it as **short recurring practice** whenever you rotate IP math — rhythm is yours, not mandated here.

Lab packages (Ubuntu VM)

```
sudo apt update
sudo apt install -y wireshark traceroute tcpdump dnsutils netcat-openbsd nmap curl
```

`wireshark` may prompt for capture permissions (`wireshark` group / capabilities) — configure once and document how you solved it.

Deliverable for this primer file

Copy the **primary resource links** into your notes tool and schedule **three** Practical Networking sections you will finish first — in order — before hopping elsewhere.

Source: `03-networking-foundations/02-osi-tcpip-tcp-udp-packet-basics.md`

Unit 02 — OSI, TCP/IP, TCP vs UDP, first captures

Theme

How the internet delivers bytes end-to-end at a layered mental model level.

Study alignment

Source	Sections
Practical Networking	OSI Model, TCP/IP Model, TCP vs UDP
Selected CCNA fundamentals	Introduction, OSI, TCP/IP
Spot video	OSI / TCP-UDP recap if confused

Ubuntu drills

Run and interpret succinctly:

```
ping -c 4 google.com
tracpath google.com
curl https://example.com/
ss -tulpn | head -40
```

Wireshark drills

Filters to practise (`tcp`), then revisit a browse session looking for `SYN`, `SYN-ACK`, `ACK` handshake segments on a benign site you authorize.

Ethics reminder: capture only labs you own authorization for.

Checklist vocabulary

OSI layers (practical—not trivia regurgitation), TCP/IP shorthand, segmentation vs reliability, ICMP echo vs TCP, TTL intuition, fragmentation/MTU as a troubleshooting axis.

Learning outcome

You can narrate handshake establishment and articulate what UDP forfeits versus TCP without hand-waving beyond your evidence.

Source: `03-networking-foundations/03-ip-addressing-subnetting-cidr.md`

Unit 03 — IP addressing, subnetting, CIDR

Theme

How hosts are numbered and partitioned.

Study alignment

Source	Sections
Practical Networking	IP Addressing, CIDR, Subnetting
CCNA fundamentals (selected)	Subnetting, CIDR deep enough to reason
Subnetting practice	https://subnettingpractice.com/ — maintain short daily sets

Ubuntu drills

```
ip a
ip route
ip -6 addr show
```

Interpret **addresses versus prefixes**.

Drill problems (paper or scratch buffer)

Understand and annotate:

Example	Annotate
192.168.1.0/24	network, broadcast, usable host ranges, gateway intuition
10.0.0.0/8	classless meaning today
172.16.0.0/16	private RFC1918 rationale

Explain in your own words: **network address, host, broadcast, meaning of /24**.

Topics

Private vs public IPv4 framing, baseline IPv6 awareness (dual-stack reality), gateway choice, overlapping/overlong prefixes as a conceptual misconfiguration trap.

Learning outcome

You can answer “how many usable hosts?” and justify a carve without calculator dependency by the end of the unit—not necessarily instant mental math flawless, but structurally competent.

Source: 03-networking-foundations/04-dns-dhcp-nat-arp.md

Unit 04 — DNS, DHCP, NAT, ARP

Theme

How systems locate peers and negotiate addresses on LAN/WAN borders.

Study alignment

Source	Focus
Practical Networking	DNS, DHCP, NAT, ARP
Selected CCNA	DNS operational basics, DHCP, NAT classifications conceptually

Ubuntu drills

Resolver checks:

```
dig google.com +short
nslookup google.com
host google.com
ip neigh show
```

Wireshark drills

Capture **DNS** lookups during a benign browse session (dns display filter vocabulary). Correlate Question/Answer with what dig returned.

Checklist vocabulary

FQDN/resolver recursion vs stub, TTL on records, DHCP offer/ack sequence at high level (not vendor exam trivia), NAT inside→outside tuple rewriting intuition,ARP/NDP neighbour discovery purpose.

Learning outcome

Given a symptom (“name resolves inconsistently”), you articulate whether to suspect resolver path, captive portal/DHCP starvation class issues, stale ARP, or NAT/port mapping quirks—then pick the next factual measurement.

Source: 03-networking-foundations/05-routing-path-model.md

Unit 05 — Routing and packet path narratives

Theme

How a datagram hops from laptop to internet service provider to remote server logically.

Study alignment

Source	Sections
Practical Networking	Routing, default gateway discourse
CCNA fundamentals (selected)	Routing introduction only

Ubuntu drills

```
ip route
traceroute google.com
tracepath google.com
```

Interpret **hop growth**—where latency spikes first.

Narrative model (produce your sketch)

Produce a handwritten or diagrammable story:

```
Laptop → LAN router/NAT → ISP → Internet ↔ remote network → destination host
```

Label where **routing decision**, **NAT rewrite**, potential **VPN tunnel** would appear if VPN later overlays.

Checklist vocabulary

Default route, longest-prefix match intuition, asymmetric routing caveat, TTL decrement as rough hop counter anecdote—not deep BGP policy.

Learning outcome

When ping works but traceroute stalls at hop *k*, you can hypothesize plausible classes of causes and what extra evidence distinguishes them—not instant diagnosis perfection.

Source: 03-networking-foundations/06-http-https-tls.md

Unit 06 — HTTP, HTTPS, TLS handshake

Theme

How browsers speak to TLS-terminated web servers.

Study alignment

Supplemental videos	Conditional
“HTTPS explained” / “TLS handshake” style NetworkChuck or equivalent	Only if plaintext vs encrypted framing still fuzzy

Ubuntu / OpenSSL drills

```
curl -I https://example.com

echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null \
| openssl x509 -noout -issuer -subject -dates
```

Adapt `example.com` to a stable host your lab can reach ethically.

Observe:

- Issuer hierarchy sense
- Validity intervals
- **SNI** necessity on shared hosting patterns

Repeat `openssl s_client` interactively reading handshake stage lines your build prints.

Browser developer tools exercise

Inspect at least:

- Response status line
- Notable security headers framing (even if benign site omits hardened set)

Topics mapping

Plain HTTP verbs & status families, redirection chains, symmetric vs asymmetric stage at high level during TLS handshake story, certificates vs ephemeral session keys shorthand.

Learning outcome

Explain end-to-end: resolve name → TCP 443 → ClientHello clues → negotiated parameters class → encrypted HTTP semantics—still coarse, factually tethered.

Source: `03-networking-foundations/07-ports-services-socket-basics.md`

Unit 07 — Ports, services, fingerprints

Theme

Associate common listening ports with service intent and tooling.

TryHackMe

Complete **Network Fundamentals** milestone tasks not yet crossed off unless already satisfied honestly.

Ubuntu drills

```
sudo ss -tulpn | head -40
sudo nmap -sV localhost
```

(Optional narrow scan.) Do **not** scan third-party hosts without explicit permission.

Memorization targets (purpose, not trivia)

Associate services you will collide with routinely:

22 SSH • 53 DNS • 80 HTTP • 443 HTTPS • **445 SMB** • **389 LDAP** • **88 Kerberos** • **3389 RDP**

Understand these as **patterns** attackers enumerate—not flashcards devoid of semantics.

netcat/SSH micro drills

Demonstrate purposeful local loopback chatter or SSH banner grab on your VMs only.

Learning outcome

When `ss` shows `LISTEN` on surprising port → you classify benign vs suspicious next-step triage thoughtfully.

Source: `03-networking-foundations/08-packet-capture-analysis.md`

Unit 08 — Packet capture (`tcpdump` + Wireshark)

Theme

Convert frames into evidence-backed hypotheses.

Ubuntu — `tcpdump` basics

Start with narrowly scoped captures on loopback while exercising a local benign service you control (permission model matters):

```
sudo tcpdump -ni lo -c20 'tcp port 80'
```

Iterate filters: broaden and narrow purposely.

Wireshark refinement

Sequences:

Pass	Goal
<code>tcp</code> filter	Isolate conversation stream
<code>dns</code> transitions	Correlate lookups
<code>http</code> baseline	Inspect cleartext legacy traffic only in sanctioned labs

TryHackMe

Finish lingering **Network Fundamentals** labs focusing on PCAP interpretation—not speed-running clicks.

Learning outcome

You can annotate a twenty-packet excerpt: handshake, TLS start (if viewed at edge instrumentation limitations), DNS preceding connect, teardown flags—truthful to what’s visible—not speculative Hollywood narrative.

Unit 09 — Integration: closed-book narrative drill

Theme

Rebuild the **full benign browser fetch** narrative without referencing notes during the narration window.

Exercise (then verify notes)

Triggered by typing `https://example.com/` mentally (adapt host ethically accessible):

Enumerate **ordered** milestones you expect:

DNS → iterative/recursive intuition → answer selection

→ destination IP choice → local routing lookup

→ outbound NAT rewrite concept → path hops class

→ TCP handshake synopsis

→ TLS ClientHello/high-level parameter negotiation story

→ HTTP request/response first line framing

Deliver **verbally aloud** once; jot gaps discovered afterward only.

Checklist competency targets

Able to crisply articulate:

TCP vs UDP contrasts in one sentence each pragmatically usable

NAT purpose & breakage symptom class

subnet mask / prefix meaning

routing vs switching functional boundary at this learning depth

capturing ethically & filtering effectively

basic HTTP semantics & TLS role

Self-grade rubric

Grade	Indicator
Pass	Complete chain with ≤2 sanctioned peek corrections
Retry	>2 foundational gaps surfaced — loop targeted micro-units

Deliverable

Write **five questions** you’d ask to debug a friend’s “internet works partly” ambiguous report using only tools/ commands from prior units—no new tools invented.

Phase — 04-cyber-security-presecurity-foundations

Directory: `04-cyber-security-presecurity-foundations/`

Source: `04-cyber-security-presecurity-foundations/01-primary-resources-environment.md`

Phase three — Cyber security foundations & PreSecurity (resources & posture)

Finish this phase understanding **how offensive and defensive work differ**, baseline **web mechanics from an engineer's vantage**, foundational **Linux and Windows ergonomics, virtualization/cloud vocabulary**, client-server decomposition, attack surface instincts, core vocabulary—not yet “weaponized exploit artistry.”

Outcome shift:

Weak framing	Strong framing
Jump to payloads	Automatically ask where brittle trust boundaries sit

Primary syllabus — TryHackMe Pre Security

Path outline — verify current module naming on the platform yourself (names drift):

[Pre Security Learning Path — TryHackMe](#)

Work the path **completionist** versus your honest pacing—integrity over badge speed.

Practice environment essentials

Dual VM posture where feasible:

Asset	Goal
Ubuntu	Commands + prior Linux phase reinforcement skim
Windows VM	Fundamental admin introspection tooling

Browsers (**Chrome/Firefox**):

- Developer tools (F12)
- Network tab (headers/status/cookies interplay)

Unit map (this numbered area folder)

Proceed through `02 → 09` sequentially unless you objectively validate earlier mastery.

Honour / ethics reinforcement

Enumerate only systems you authorize (owned VMs / THM / contracted ranges). Maintain lab journals without leaking reusable third-party-sensitive artifacts.

Source: `04-cyber-security-presecurity-foundations/02-introduction-disciplines.md`

Unit 02 — Introduction to cybersecurity disciplines

Theme

Map professional lanes without cargo-cult stereotypes.

TryHackMe — PreSecurity (early modules)

Locate **Introduction** / **primer** grouping—modules approximating:

- Intro to Cyber Security buckets
- Offensive vs defensive orientations

(Exact wording changes—trust the syllabus map on-platform.)

Vocabulary checkpoints

SOC patterns, penetration testing posture vs AppSec/App engineering hardening overlaps, vuln vs risk shorthand, incident response lifecycle naming only at awareness depth.

Writing drill (notebook)

Without references, produce working definitions—in your tone—for:

Penetration tester vs security engineer (build+defend blended org reality) vs dedicated defensive tooling engineer archetype.

Contrast **pure red teaming theatre** caricature vs iterative technical assurance realistically.

Learning outcome

You stop collapsing “hack” into vagueness—you name **activity classes** responsibly.

Source: `04-cyber-security-presecurity-foundations/03-networking-security-perspective.md`

Unit 03 — Networking through a security mindset

Theme

Re-frame Phase two facts as **boundary & visibility** problems.

TryHackMe — Network Fundamentals (PreSecurity-aligned segment)

Redo or validate modules around:

LAN vs broader segmentation intuition

OSI / packets vs frames mnemonic utility

Extend / route / boundary abstractions pedagogically surfaced there

Trust the authoritative module catalogue on-platform.

Ubuntu refresh (short)

```
ping example.com -c 3
ip a
ip route
ss -tulpn | head -30
```

Narrative exercise

Rebuild—without peeking—the chain:

browser → DNS → routing → TCP → server port binding

Identify **trust boundaries** attackers love (resolver lying class, rogue gateway class, interception—not detailed attacks yet).

Learning outcome

You instinctively annotate attacker opportunities at each hop abstractly—even before exploiting.

Source: `04-cyber-security-presecurity-foundations/04-how-the-web-works-security-view.md`

Unit 04 — How the web works (engineering + security vantage)

Theme

Interpret HTTP-era surfaces before diving offensive labs later.

TryHackMe — How The Web Works (cluster)

Work modules approximating detailed DNS traversal, granular HTTP anatomy, lifecycle “putting it together” syntheses—adapt to refreshed naming.

Browser DevTools — **Network** tab session:

Observe at least:

- Selected request headers & response baseline
- Status code lineage on redirect chain benign site
- Cookie emission & scope concepts (don’t memorize vendor quirks early)

Guided observation

Navigate an intentionally simple static site versus a heavyweight SPA minimally—feel difference in chatter volume—not deep front-end apprenticeship.

Learning outcome

You explain where application trust incorrectly assumes sanitized client assertions—foundationally.

Source: `04-cyber-security-presecurity-foundations/05-computer-virtualization-cloud-basics.md`

Unit 05 — Computers, virtualization, cloud (compressed systems lens)

Theme

Machines abstracted—not electrical engineering depth.

TryHackMe — Computer Fundamentals block

Approximate thematic coverage usually titled around:

computer internals overview, architectural types taxonomy, basic client/server, virtualization primer, foundational cloud motifs

(Section titling rotates—mirror live course.)

Conceptual layering drill

Produce a schematic:

Browser → logical web tier → persistence tier

Then overlay ephemeral vs durable failure surfaces.

Write one paragraph distinguishing **containers vs VMs** accountability tradeoffs succinctly—even if imperfect precision—then refine via official docs later deliberately.

Learning outcome

Architectural shorthand stops being buzzword bingo—you narrate plausible component boundaries responsibly.

Source: 04-cyber-security-presecurity-foundations/06-linux-fundamentals-security-skim.md

Unit 06 — Linux fundamentals skim (security angle)

Theme

Harvest parallel reinforcement—don't relive beginner grind fully.

Constraint: assuming Phase one Linux completeness **honestly**, skim TryHackMe **Linux Fundamentals** arcs where they appear — prioritize deltas & security glimpses—not re-solving trivial tasks unless memory gaps surface.

Ubuntu micro refresh:

```
ps aux | head
top -b -n1 | head
journalctl --since "30 min ago" | tail -n 40 || true # shorten window if noisy
chmod --help >/dev/null
ssh -V 2>/dev/null || true
curl -I https://example.com
```

Learning outcome

You consolidate identity of processes, persistence-location classes, and mutable configuration surfaces from a security lens—not another generic shell tutorial pass.

Source: 04-cyber-security-presecurity-foundations/07-windows-fundamentals-security.md

Unit 07 — Windows fundamentals for security newcomers

Theme

Operational literacy on Microsoft's surface area.

TryHackMe clusters

Sequential Windows fundamentals modules numbered typically 1-3 (titling rotates).

VM exercises — PowerShell

```
Get-Process | Select-Object -First 15
Get-Service | Where-Object Status -EQ 'Running' | Select-Object -First 15
hostname
whoami
```

Classic shell auxiliaries still matter:

```
tasklist
ipconfig /all
net user
```

Focus lenses

Filesystem ACL intuition, registry mention only as hive-of-misconfigs placeholder, Windows services escalation class awareness without exploit detail yet, **UAC** high-level—not bypass catalog.

Learning outcome

You stop treating Windows internals as unknowable blobs—you ask structured triage questions about identity, privileges, services, exposures.

Source: `04-cyber-security-presecurity-foundations/08-integration-drills-architecture-narratives.md`

Unit 08 — Integration drills (architecture narratives)

Theme

Chain earlier abstractions verbally.

Exercise A — Narrate end-to-end (no refs during attempt):

DNS → TLS-wrapped HTTP → application → database-ish persistence → response egress shaping

Exercise B — Map:

client credentials presentation → verification → authorization mediation

Mark **common fragility hypotheses** succinctly—not exploit recipes.

Retro scoring

Enumerate **explicit mistakes** corrected post-check vs blind spots lingering.

Learning outcome

You produce **layered causal stories**, not acronym soup regurgitated.

Source: `04-cyber-security-presecurity-foundations/09-owasp-mindset-introduction.md`

Unit 09 — OWASP mindset introduction

Theme

Industrialize probing questions—not yet mastery of every CWE.

Canonical references (orientation passes)

Browse structure & headings (do not memorize unsorted lists shallowly):

- [OWASP Top 10 Overview](#)

Themes to absorb

Risk vs vuln distinctions (sloppy shorthand dangerous), Attack surface enumeration habits, Misconfiguration subclass consciousness, Principle of least privilege, Defense in depth, CIA triad as coarse lens—not religious dogma.

Practice interrogation scaffold

Whenever observing a subsystem (even benign personal project), consciously ask subsets:

Authentication? Authorization granularity? Dangerous trusts on input? Secrets lifecycle realism? Operational logging usefulness? Dependency posture?

Annotate unanswered unknowns—not immediate fixes required.

Learning outcome

Thinking acquires habitual **challenge posture**—not performative jargon drops only.

Phase — 05-web-backend-auth-security-foundations

Directory: 05-web-backend-auth-security-foundations/

Source: 05-web-backend-auth-security-foundations/01-primary-resources-and-lab-environment.md

Phase four — Web, backend & auth security (resources & lab posture)

- Objective: reproducibly articulate **attacks**, **mitigations**, and pragmatic **reviews** touching modern backends—not performing shallow exploit cosplay devoid of taxonomy.
- Desired capability snapshot:
- Reproduce cornerstone web vuln classes ethically in **local**/PortSwigger/authorized arenas
 - Articulate defenses & friction classes
 - Operate **Burp Proxy** thoughtfully
 - Comfortably discuss **JWT lifecycle pitfalls** without magical thinking
 - Interpret **OAuth** redirect & token handling failure families
 - Orient **API security & dependency/supply-chain** tooling stories
 - Honor **review mental model**, not leaderboard chasing alone.

Core references

Corpus	Purpose
PortSwigger Web Security Academy	Hands-on thematic labs spanning injection, SSRF, access control nuances, deserialization, JWT, OAuth, request smuggling intro, XSS family, CSRF, file path traversal & upload pitfalls, advanced optional edges
OWASP Top 10 Project	Category mapping fidelity
OWASP API Security Top 10	Backend API exposure reality
Burp Suite	interception / repeater ergonomics mastery
Trivy (image & dependency scanning illustrative)	container supply posture orientation

Ethics: constrain traffic to sanctioned environments / personal builds / Academy targets.

Local technical stack cues (adapt to your codebase)

- Symphony project with realistic auth surfaces, JWT experiments, OAuth client/server flows consciously isolated, Dockerized dependencies, reproducible seeded data—not production secrets leakage.
- Phase rule (each vulnerability class minimally):
- Theory → Controlled attack rehearsal → Mitigation/defense rationale → Sanity pass on personal project surfaces if applicable.
- Units 02–12 operationalize sequentially.

Source: 05-web-backend-auth-security-foundations/02-http-and-burp-basics.md

Unit 02 — HTTP in depth via Burp basics

Theme

Operational HTTP literacy through deliberate interception.

PortSwigger: primer units on HTTP request/response grammar, verbs, evolving header roles, cookie mechanics, numeric status taxonomy—progress until Proxy confidence solid.

Burp competencies:

Activate **Proxy intercept** ethically on local target only.

Explore **Repeater** iteration discipline.

Harvest **history** selectively—avoid hoarding unrelated sensitive noise.

Symfony/local API micro mutation:

```
GET /users?page=1
```

Iterate parameter boundary tests you hypothesize—not random fuzz spam without intent.

Topics: verbs, redirection trust, HTTPS termination awareness, framing future abuse avenues abstractly only.

Source: 05-web-backend-auth-security-foundations/03-sql-injection-and-defense.md

Unit 03 — SQL injection & defensive fundamentals

Academy SQLi progression: union/error/boolean/blind/time-delay families—ethical labs only.

Defensive juxtaposition mentally:

Dangerous hypothetical raw concatenation snippet vs parameterized / ORM disciplined parameter binding stance.

Doctrine (or your stack) auditing drill:

Locate dynamic SQL—even “small” shortcuts.

Classify parameterized vs interpolated risk.

Produce remedial refactor sketch even if postponing rollout.

Topics: inference channels, mitigation taxonomy (prepared statements, least privilege DB accounts, hardened error surfaces).

Source: 05-web-backend-auth-security-foundations/04-xss-and-csp-basics.md

Unit 04 — XSS, CSP, and output-context discipline

Labs: reflected, stored, and DOM XSS arcs; SSTI introductions when your Academy path exposes them—template injection boundaries bleed into careless dynamic evaluation.

Defense lenses:

- Context-aware escaping (HTML vs attribute vs JS vs URL—not one magic sanitizer).
- CSP as **layered friction**, not a single silver directive paste.

Starter header awareness: CSP tightening tradeoffs; frame embedding controls (X-Frame-Options / frame-ancestors migration mental map); referrer policy narrowing cautiously.

Local audit prompt

Locate template `| raw` / unescaped bridging in Symfony/Twig (or equivalents). Classify misuse risk tier honestly.

Source: `05-web-backend-auth-security-foundations/05-csrf-sessions-authentication-flows.md`

Unit 05 — Sessions, authentication hardening & CSRF

Academy arcs: Authentication vulnerability cluster + CSRF patterns.

Themes: brute-force rate friction, MFA absence implications, flawed password reset choreography classes, fixation risks, logout invalidation fidelity, predictable tokens, predictable sessions.

Mitigation shorthand: CSRF synchronization token patterns (SameSite cookie augmentation mental model—not sole defense fantasy).

Symfony audit angles: integrated CSRF token usage on state-changing routes, lifecycle & rotation strategies, sane idle & absolute timeouts if applicable.

Source: `05-web-backend-auth-security-foundations/06-jwt-security-lifecycle.md`

Unit 06 — JWT security & verification discipline

Labs: weak secrets, naive alg confusion classes, omission of signature enforcement stories—Academy-guided only.

Operational checklist mental model claims iss, aud, expirations/skew defenses, JWKS rollover tolerance, revocation partial realities.

Bridging hygiene: bcrypt/argon adaptive hashing at password ingress edges—distinct topic but intersects naive token piggybacking anti-patterns.

Local inspection: JWT middleware validation fidelity on sample tokens you mint intentionally flawed in dev.

Source: `05-web-backend-auth-security-foundations/07-oauth-oidc-and-pkce.md`

Unit 07 — OAuth 2 / OIDC attack surfaces

Labs: lax redirect URIs, state omission, leakage via referrers/history/logging, brittle implicit-era anti-patterns if modules still illustrative.

Comfort narrative (no exam trivia): Authorization Code + PKCE for public clients rationale, refresh token stewardship, whitelist discipline for redirect URIs, careful logging redaction stance.

Symfony integration audit: OAuth client configs & callback verification rigor—even stub environments.

Source: `05-web-backend-auth-security-foundations/08-idor-and-access-control-ownership.md`

Unit 08 — Access control, ownership & IDOR

Labs: horizontally & vertically scrambled object reference scenarios—ethical environments only.

Practical heuristic: mutate identifiers with intent (/invoice/5 → neighbouring IDs) documenting authorization outcomes—not blind hammering blindly on production.

Symfony mental tools: voters, policy checks, verifying domain ownership—not controller existence alone mistaken for authorization truth.

Source: `05-web-backend-auth-security-foundations/09-ssrf-uploads-deserialization.md`

Unit 09 — SSRF, uploads & deserialization pitfalls

Themes (labs):

SSRF chaining classes & metadata credential exfiltration folklore responsibly studied only in sanctioned contexts.

Upload validation—not extension alone fantasies nor naive MIME trust.

Unsafe deserialization echoes & race-adjacent classes introduced where Academy modules exist—observe timing/lifecycle caveats ethically.

Source: `05-web-backend-auth-security-foundations/10-api-security-dependency-scanning.md`

Unit 10 — OWASP API Top 10 & dependency posture

Work [OWASP API Security Top 10](#) with mapping mental exercises against your hypothetical RESTish JSON surfaces—even if illustrative.

Operationalize scanning cadence introductions:

composer audit rhythms (supply chain caveat: advisories incompleteness)

npm audit parallelism if polyglot

Trivy on container images selectively for teaching—not production gate absolutism prematurely

SBOM conceptual orientation (SPDX / CycloneDX names) without bureaucracy cosplay devoid of actionable next steps choice.

Source: `05-web-backend-auth-security-foundations/11-integration-local-security-review.md`

Unit 11 — Consolidated deliberate local assessment

Forge minimal toy API locally embedding:

JWT issuance & validation quirks you control

OAuth client dance stub

Controlled file-upload endpoint hardened iteratively

Role separation surface

Enumerate findings after Burp-guided passes & Academy skill cross-check—not vanity scores.

Remediation backlog: categorize quick wins versus structural rework candidates.

Source: `05-web-backend-auth-security-foundations/12-owasp-top-ten-mapping-deep-dive.md`

Unit 12 — OWASP taxonomy mapping rehearsal

Produce **personal** mapping rehearsal (adapt naming to current Top 10 cycle if OWASP numbering shifts—verify headings online):

Examples of mapping style (pattern—not immutable gospel):

SQLi ⇒ Injection class

Dominant XSS classes ⇒ Injection (context nuance annotate)

Broken auth/JWT ⇒ Authentication failures cluster

Broken OAuth redirect discipline ⇒ Broken access control overlaps + authentication choreography failures

Dependency findings ⇒ Vulnerable components

SSRF ⇒ dedicated SSRF row / server-side abuse cluster

Produce per-vulnerability rehearsal row:

Attack intuition → Detective signal framing → Mitigation strata → OWASP anchor → Local reproduction pointer (ethical)

Expect iteration—this spreadsheet mentality matures—not one-and-done cram.

Phase — 06-burp-http-security-testing-workflow

Directory: `06-burp-http-security-testing-workflow/`

Source: `06-burp-http-security-testing-workflow/01-resources-and-practice-environment.md`

Burp Suite, HTTP analysis & security-testing workflow — resources & environment

Outcome: you wield **Burp Suite as a repeatable debugging harness**—intercept, reshape, analyse **JWT OAuth HTTP API** traffic on systems you authorize—not aimless clicking through tabs.

Anti-pattern	Target pattern
Memorise tab names without intent	Structured intercept → analyse → mutate → replay

Courses (ordering matters)

Apprentice (first)

- [Burp Suite Apprentice — Web App Pen Testing](#)

Practitioner (after Apprentice comfort)

- [Burp Suite Practitioner — Web App Pen Testing](#)

Treat Udemy syllabus sections as movable if the vendor restructures pages—anchor on **skills**, not timestamps.

Practice labs parallel track

Every Academy theme you practise should see **paired Burp ergonomics**:

- [PortSwigger Web Security Academy](#)

Installer

- [Burp Suite download](#) — **Community** is adequate for foundational mechanics; note which workflows require paid features and substitute **curl/Repeater-only** rehearsals ethically.

Local stack (authorised only)

Symfony (or comparable) demo API exposing **JWT** + **OAuth** flows you control

Browser Developer Tools (**Network**)

Trusted burp-proxy CA for your lab browser profile

Intercept **only hosts you own**.

Honour / ethics reminder

Laboratory artefacts only—respect OffSec assessment policy, CSP on third-party sites, contractual scopes.

Source: `06-burp-http-security-testing-workflow/02-proxy-intercept-and-history.md`

Unit 02 — Proxy, interception & HTTP History

Theme: grounding Burp circulation—**traffic you may legally inspect**.

Course alignment: Apprentice instalation → browser → imported CA → Proxy tabs.

Operational targets:

- Understand **Intercept on/off**, scope filters, unintended leakage of unrelated tabs.
- **HTTP History** triage—not hoarding—but correlating benign mutation attempts.

Symfony drill:

```
GET /api/users?page=1
```

Mutate **page** values you hypothesise (**e.g. 500**), observe behavioural deltas defensively—not blind numeric spam absent hypothesis.

Topics: proxies, interception cadence, request/response halves, baseline headers vs cookies interplay.

Source: `06-burp-http-security-testing-workflow/03-repeater-and-authorisation-probes.md`

Unit 03 — Repeater-driven HTTP manipulation

Continue Apprentice sections on **Repeater**.

Laboratory rehearsals (ethical targets only):

Adjust **Cookie**, **Authorization** headers, brittle numeric identifiers deliberately:

```
GET /profile?id=SELF  
→ GET /profile?id=SIBLING_SAFE_ID  
GET /invoice/5 → GET /invoice/6    # authorised synthetic fixtures only
```

Map outcomes to hypothesis classes (authorisation leakage vs benign 404 distinctions).

Symfony targets locally: JWT minting endpoints, OAuth dance callbacks, coarse role segregation routes.

Themes: iterative tampering hygiene, duplication discipline, documenting unexpected successes **without leaking secrets externally**.

Source: `06-burp-http-security-testing-workflow/04-decoder-comparer-and-jwt-surface-analysis.md`

Unit 04 — Decoder, Comparer & JWT structural literacy

Completing Apprentice Decoder + Comparer arcs.

Laboratory JWT (issuance you govern):

Inspect JSON claims such as illustrative:

```
{ "sub": "123", "role": "admin" }
```

Never reuse production tokens—the exercise is comprehension of claim semantics & encoding layers (Base64url segments)—not glamourising secrets.

Discuss **automatic header rewriting** responsibly (Burp macros / Match rules where edition permits)—community edition may constrain; adapt with scripted curl if needed.

Symfony drill dimensions: tempered **exp**, **iss**, **aud** mutation attempts + signature acceptance outcomes you document factually without bypass cheerleading fiction.

Source: 06-burp-http-security-testing-workflow/05-intruder-throttling-awareness.md

Unit 05 — Intruder basics & responsible automation stance

Advance into **Practitioner** Intruder teachings—only after repeatable manual manipulations stabilise.

Local login form sandbox you **own**:

- Placeholder positions, minimal payload palettes, disciplined **grep-based response diffing**.
- Consciously articulate **why blind brute-force** is forbidden on foreign infrastructure.

Observe **rate-limit / lockout artefacts** ethically on your seeded accounts only.

Themes: payloads, concurrency tradeoffs (lab-scale), aligning automation with hypotheses—not spray-and-pray amorality camouflaged as “practice”.

Source: 06-burp-http-security-testing-workflow/06-authentication-and-session-behaviour.md

Unit 06 — Authentication & session artefacts in motion

Parallel PortSwigger **Authentication + Sessions** lab clusters alongside **intercepted Symfony flows**.

Harvest (under authorisation):

- Lifecycle of session cookies (**flags, scopes**, rotation cadence hypotheses).
- Header vs cookie token transport inconsistencies.

Reason about MFA absence implications, brittle refresh cadence exposures, fixation class mistakes—conceptual depth first.

Deliverable snippet: enumerated **signals** distinguishing healthy vs suspicious session choreography on your sandbox metrics logs.

Source: 06-burp-http-security-testing-workflow/07-oauth-flow-and-token-intercept.md

Unit 07 — JWT + OAuth flow interception drill

Reuse Academy JWT + OAuth exercises while mirroring interception in **authorised Symfony OAuth** playgrounds.

Observe **authorise** ⇒ **callback** choreography:

`/oauth/authorize` query parameters (**client_id, redirect_uri, state**)

Captured **authorization code hop** ⇒ token exchange artefacts

Document **misconfiguration hypotheses** tied to lax redirect allowances or absent **PKCE** when public clients analogue appears.

Source: 06-burp-http-security-testing-workflow/08-json-and-header-manipulation-apis.md

Unit 08 — Structured body tampering & API debugging ergonomics

Focus: **Repeater + Proxy interplay** enriching JSON payloads you control ethically.

Manipulate illustrative JSON:

```
{ "userId": 5 } → { "userId": 6 }
```

Record authorisation divergence patterns & API error fidelity.

Themes: differentiated error surfaces, leaky stack traces versus disciplined API failure vocabulary, defensive logging blind spots glimpsed indirectly.

Source: 06-burp-http-security-testing-workflow/09-integration-burp-review-pipeline.md

Unit 09 — Consolidated authorised API review choreography

Elevate miniature Symphony surface featuring **JWT + OAuth + benign upload façade + differentiated roles**.

Run deliberate pipeline narration:

Request capture → Scoped intercept → Controlled modification → Repeatable experimentation → Structured response appraisal → Lightweight security review jot

Enumerate **risk classes witnessed**—not vanity vulnerability counts—for your lab only.

Source: 06-burp-http-security-testing-workflow/10-question-checklist-before-advancing-requests.md

Unit 10 — OWASP-aligned interrogations through Burp ergonomics

For each materially interesting request replayed ethically, habitualise mentally:

Lens	Probe
Access control drift	Numeric / UUID boundary surprises? Horizontal creep?
Rate limiting realism	Exhaustion amplification responsible only in sandbox
Session fragility	Rotation? fixation class exposures? fixation replay?
JWT posture	Audience & issuer coherence? Tamper reaction clarity?
Sensitive data egress	Overshared payloads or headers mirrored downstream?
Input validation seams	Parsing anomalies under structural edits

This scaffold complements OWASP cheat-sheets—you supply judgement, Burp surfaces evidence.

Phase — 07-windows-active-directory-enterprise

Directory: 07-windows-active-directory-enterprise/

Source: 07-windows-active-directory-enterprise/01-resources-and-environment.md

Windows, Active Directory & enterprise authentication — resources & posture

Desired perspective: articulate **why** consolidated enterprise identity artefacts behave—not exploit trophy hunting prematurely.

Anti-pattern vs goal:

Drift	North star
Memorise flashy AD exploits	Explain trust boundaries impacting availability & risk

Primary platforms (names evolve—mirror live catalogues):

- [TryHackMe](#) — sequential **Windows Fundamentals 1-3** modules + curated **Active Directory Basics** introductions.
- [Hack The Box Academy](#) — **Windows fundamentals**, AD fundamentals corridors, foundational **SMB**, **LDAP**, **Kerberos** exposition.

Lab necessities

Isolation-controlled **Windows** VM paired with reachable **Ubuntu** workstation for tooling diversity.

Comfort targets: coherent **CMD** primitives + escalating **PowerShell** inspection habits.

Source: 07-windows-active-directory-enterprise/02-windows-operating-environment-basics.md

Unit 02 — Windows system fundamentals recap

Operational drills (elevated responsibly inside your VMs):

PowerShell glimpses (`Get-Process`, `Get-Service`, `hostname`, `whoami`, `ipconfig /all`)

Classic shell companions (`tasklist`, `net user`, `hostname`).

Enumerate baseline mental map: filesystem layout intuition, servicing model, rudimentary ACL awareness, interactive vs noninteractive session nuance—no exploit catalogues yet.

Source: 07-windows-active-directory-enterprise/03-local-security-surfaces-logging.md

Unit 03 — Permissions, logging posture & Defender awareness

Leverage WF2 themes (titling rotates on THM catalogue).

Expose **Get-LocalUser** / **Get-LocalGroup** surfaces.

Sample **Security** / **System logs** selectively—volume managed—articulate what **success vs failure** audit entries imply—not raw log drowning.

Discuss NTFS discretionary ACL interplay with service identities at conceptual depth.

Source: 07-windows-active-directory-enterprise/04-powershell-triage-literacy.md

Unit 04 — PowerShell discipline for auditors

Structured exercises:

Get-ChildItem, Get-Content, Get-Service, Get-Process, Select-String

Composition rehearsal:

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 15
```

Aim for **repeatable scripted micro-queries** aiding triage—not one-off improvised typing alone.

Source: 07-windows-active-directory-enterprise/05-active-directory-structure-gpo-overview.md

Unit 05 — Active Directory structural literacy

Hybrid THM AD Basics + Academy AD fundamentals exposition.

Articulate hierarchically:

Company domain ⇒ **Organizational Units** ⇒ grouped users ⇒ **Group Policy Objects** interplay.

Produce whiteboard-able narrative connecting **authentication requests** ⇒ **directory lookup** ⇒ **authorization evaluation** scaffolding—still conceptual.

Topics: domains, controllers, authoritative replication intuition (no deep internals exam yet).

Source: 07-windows-active-directory-enterprise/06-ldap-queries-and-dns-glue.md

Unit 06 — LDAP as directory query abstraction

Explore Academy LDAP curricula: binding modes, illustrative queries distinguishing users vs groups, DNS SRV locator record purpose for AD locating services.

Analogue LDAP as typed graph queries over identity—not magical incantations.

Sanctioned practise: scripted queries pointing solely at VMs you erected.

Source: 07-windows-active-directory-enterprise/07-kerberos-ntlm-and-adcs-framing.md

Unit 07 — Kerberos vs NTLM comparison & PKI edges

Goals (lab-scoped exposition):

Kerberos ticketing outline (**TGT / TGS / KDC role / SPN concept**) juxtaposed honestly against **legacy NTLM challenge-response**.

Surface **certificate template** & organisational CA interplay only at comprehension depth—defer weaponised misconfiguration exploitation drills until authorised curricula signal readiness.

Source: 07-windows-active-directory-enterprise/08-smb-winrm-and-sysvol.md

Unit 08 — SMB, WinRM & SYSVOL intuition

Laboratory exposures only:

Understand **share enumeration** ethics boundary—never aim tools at unmanaged estate.

Discuss **SMB permission inheritance** anomalies vs Linux mount mentalism.

Sketch **WinRM** intent (management channel) distinguishing HTTP listener ports academically.

Enumerate **SYSVOL / Group Policy scripting distribution** rationale & integrity expectations.

Source: 07-windows-active-directory-enterprise/09-enumeration-mindset-signals-before-exploit.md

Unit 09 — Enumeration mindset: listen to systems first

Leverage sanctioned THM enumeration rooms.

Collect **signals** ethically:

Hostname patterns, surfaced users/groups classifications, dormant services fingerprints, unintended share artefacts, latent listener ports correlations.

Articulate iterative cycle:

Observe → sane hypothesis → evidence gather → escalate analytical depth—not premature exploit leap.

Source: 07-windows-active-directory-enterprise/10-integration-enterprise-grant-chain.md

Unit 10 — Integration narrative rehearsal

Practice closed-notes storytelling:

Interactive **Windows user/session ⇒ directory membership propagation ⇒ Kerberos ticketing hop intuition ⇒ LDAP attribute lookup path ⇒ SMB resource reach ⇒ validated authorisation.**

Contrast **Kerberos delegated trust model** succinctly versus **NTLM downgrade risk class** exposures—truthful to syllabus depth attained, not melodrama.

Phase — 08-kali-linux-methodology-tools

Directory: `08-kali-linux-methodology-tools/`

Source: `08-kali-linux-methodology-tools/01-resources-environment.md`

Kali-focused toolchain & methodological validation labs — framing

Perspective shift articulated:

Kali is a disposable chassis — capabilities & disciplined methodology persist across distributions.

Anchoring course:

- [Kali Linux Hands-On Penetration Testing Labs](#)

Reference roots:

[Kali Linux](#) • [Nmap](#) • [ffuf](#) • [Gobuster](#)

Environment

Authorised constellation: **Kali** workstation + reproducible Symphony / JWT / OAuth lab stack + Dockerised extras **you govern**.

Source: `08-kali-linux-methodology-tools/02-shell-and-package-hygiene.md`

Unit 02 — Shell hygiene, journaling & repeatable labs

Treat course installation plus update cadence academically (snapshot or rollback readiness first).

Practice:

```
sudo apt update && sudo apt full-upgrade -y      # consciously; know rollback story
mkdir -p securitylab && cd securitylab
history | grep nmap                             # habitual audit echoes
grep -Rn pattern .                              # restrained scope first
curl -I https://localhost/placeholder           # benign reachability rehearsals only
```

Outcomes: literate APT usage, journaling command rationales—not blind copy-pasta.

Source: `08-kali-linux-methodology-tools/03-nmap-depth-local-targets-only.md`

Unit 03 — Authorised depth scanning with `nmap`

Operate exclusively against VMs you erected (e.g. local Symphony `php -S fixture`).

Iterate flags discussed in syllabus order—internalise sequencing:

presence sweep → versioning curiosity → scripted safe defaults escalating only with explicit approval inside your tenancy.

Articulate rationale for SYN vs connect scans at high level—not raw memorisation devoid of IDS awareness discussion on shared networks (**avoid shared Wi-Fi experiments** causing collateral noise).

Source: 08-kali-linux-methodology-tools/04-http-curl-first-debug-loop.md

Unit 04 — CURL-first reasoning on sanctioned APIs

Examine illustrative Symfony surfaces:

```
curl -sv http://localhost:8080/version
curl -X POST ... # ethically crafted bodies only on owned fixtures
curl -o /dev/null -D - ...
```

Interpret status families, deceptive redirects, chunked anomalies—pair with structured JSON pretty viewers when payloads warrant.

Source: 08-kali-linux-methodology-tools/05-gobuster-directory-discovery.md

Unit 05 — Structured directory probing with Gobuster

```
sudo apt install gobuster
gobuster dir -u http://localhost:8080 -w /usr/share/wordlists/dirb/common.txt
```

Explain why unexpected `/debug`, `/internal`, dormant `/admin` artefacts matter defensively—even when benign in your scaffold.

Source: 08-kali-linux-methodology-tools/06-fuzzing-surfaces-with-ffuf.md

Unit 06 — Precision fuzz placeholders via `ffuf`

```
sudo apt install ffuf
ffuf -u http://localhost:8080/FUZZ -w wordlist.txt
ffuf -u http://localhost:8080/api/FUZZ ...
```

Maintain **narrow wordlists**, rate discipline, conscientious concurrency—ethical labs only.
Document mis-hit patterns illuminating hidden enumerations—not merely status-code bingo.

Source: 08-kali-linux-methodology-tools/07-local-jwt-forensics.md

Unit 07 — JWT forensics aligning Burp/curl artefacts

Decode representative lab tokens illustrating **expiration**, signature enforcement outcomes, illustrative claim mutation consequences—issued only from systems you steward.

Source: 08-kali-linux-methodology-tools/08-local-oauth-observation-loop.md

Unit 08 — OAuth flows from browser vantage through proxy logs

Observe holistic chain:

browser engagement → authorised redirect choreography → ephemeral code exchange phase → bearer usage on API—all under lab consent.

Source: `08-kali-linux-methodology-tools/09-integration-lab-chain.md`

Unit 09 — Choreographed toolchain rehearsal

Controlled pipeline documentation:

Nmap sanctioned sweep → Structured HTTP probing → JWT/OAuth interplay checks → Burp corroborations → handwritten review appendix

Emphasise **repeatability** journaling—not vanity speed-run metrics.

Source: `08-kali-linux-methodology-tools/10-structured-review-checklist-extension.md`

Unit 10 — OWASP-aligned testing scaffolding after tool rehearsal

Operational snapshot mental overlay:

Enumerate → Interpret HTTP fidelity → Inspect token surfaces → Replay OWASP-aligned checklist hypotheses → summarise risk registers responsibly.

Phase — 09-enumeration-methodology-deepening

Directory: `09-enumeration-methodology-deepening/`

Source: `09-enumeration-methodology-deepening/01-resources-and-note-taking.md`

Systematic enumeration, AD tooling relationships & hardened lab methodology — resources

Integrated laboratory assets across **authorised** VMs (Kali, Ubuntu service host, reproducible simplified AD classroom if you deliberately construct one internally).

Leverage rotational platforms:

[TryHackMe](#) — enumerated / SMB / Windows / AD-aligned rooms sanctioned for study.

[Hack The Box Academy](#) — SMB, LDAP, Kerberos intros, BloodHound orientation.

Tool references (authorised contexts only):

- [Impacket](#) suite — conceptual literacy first.
- [BloodHound](#) identity relationship mapping ethos.
- [PEASS-ng](#) privilege escalation auditors—run only boxes you erected.

Note-taking ergonomics cited as **examples** — choose any structured editor respecting your privacy stance (CherryTree / Joplin / Obsidian or plain Markdown vaults).

Do **not** aim tooling at unmanaged external infrastructure—integrity & lawfulness precede ego metrics.

Source: `09-enumeration-methodology-deepening/02-nmap-methodology-internal-labs-only.md`

Unit 02 — Structured `nmap` reasoning ladders

Laboratory choreography inside owned dual-VM topography—never opportunistic wide-area sweeps.

Emphasise pipeline:

Enumerate listener facts → correlate service fingerprints → formulate the next investigative step → log evidence—not one-off commands without notes.

Source: `09-enumeration-methodology-deepening/03-ffuf-api-and-path-enumeration.md`

Unit 03 — Directed `ffuf` on local API facets

Apply tight wordlists to paths such as `/api/FUZZ` on authorised Symfony scaffolding.

Combine `curl` + JSON tooling with `ffuf` output deltas to classify anomalies deliberately.

Source: `09-enumeration-methodology-deepening/04-gobuster-and-recursive-discovery.md`

Unit 04 — Recursive content discovery drills

Operationalise Gobuster variants (dir / dns / vhost curricula as surfaced by your syllabus).

Optional: compare **feroxbuster** ergonomics locally when the course directs it—stay within hosts you erected.

Source: `09-enumeration-methodology-deepening/05-password-hash-lab-etiquette.md`

Unit 05 — bcrypt / Argon2 awareness & sanctioned cracking rehearsals

Demonstrate illustrative PHP hashing:

```
password_hash('ExampleSecret!', PASSWORD_BCRYPT);
```

Discuss work-factor tradeoffs versus Argon2 class algorithms.

Introduce **John** / **Hashcat** workflows **only on synthetic hashes you generated** inside the lab—never public breach material.

Source: `09-enumeration-methodology-deepening/06-smb-shares-with-smbclient.md`

Unit 06 — SMB share visibility on owned Windows VMs

```
sudo apt install smbclient  
smbclient -L localhost -U guest # illustrative; adapt to authorised lab credentials
```

Document share naming patterns, DACL inheritance surprises, unintended world-readable artefacts—ethical disclosure mindset even when practising alone.

Source: `09-enumeration-methodology-deepening/07-enum4linux-style-signal-harvest.md`

Unit 07 — `enum4linux` & LDAP-adjacent triage etiquette

Install per distro guidance (`sudo apt install enum4linux` etc.) — **targets must remain systems you erected**.

Harvest **sanitised** summaries: users/group hints, SMB pipe enumerations glimpses—paired with handwritten hypothesis notes.

Cross-link early LDAP rehearsal (`ldapsearch` style queries) pointing only inward.

Source: `09-enumeration-methodology-deepening/08-impacket-high-level-orientation.md`

Unit 08 — Impacket primitives as protocol teaching aids

Treat [Impacket](#) tooling as **lesson lenses** illuminating SMB / LDAP / Kerberos interplay—not production wrecking shortcuts.

Discuss representative scripts (`GetUserSPNs.py` , SMB client wrappers) ethically in classroom ranges only.

Source: `09-enumeration-methodology-deepening/09-bloodhound-trust-relationship-reading.md`

Unit 09 — BloodHound relational literacy

Leverage sanctioned Academy pathways and official [BloodHound documentation](#) orientation.

Analytic pivot:

Away from caricature questions (“single mega-admin tag”) toward **delegation graphs** showing who trusts whom—which informs both attack hypothesis *and* hardening backlog reading.

Only ingest data from laboratories you erected and cleared for graph export.

Source: `09-enumeration-methodology-deepening/10-peass-local-configuration-review.md`

Unit 10 — PEASS auditors for configuration archaeology

Use selective [linPEAS](#) / [WinPEAS](#) runs **only on throwaway VMs** you own—not production.

Harvest categories: odd service identities, brittle scheduled tasks, interesting environment variables, Linux SUID artefacts, Windows mis-set ACL pockets—then write defensible remediation notes **for lab closure**.

Source: `09-enumeration-methodology-deepening/11-integration-methodology-rehearsal.md`

Unit 11 — End-to-end methodology rehearsal

Practice narrating—not executing blindly:

Fingerprint listeners → corroborate service roles → pivot protocol class (HTTP / SMB / LDAP / Kerberos hypotheses) → gather bounded evidence packs → formulate validation experiment → summarise documentation bundle

Emphasise **documentation rhythm** contemporaneous—not retroactive scribbling devoid of fidelity.

Source: `09-enumeration-methodology-deepening/12-asvs-api-security-perspective-for-backend-stack.md`

Unit 12 — Symphony-flavoured ASVS / API security deepening

Cross-read authoritative sources—verify current revision levels yourself:

[OWASP ASVS](#) (architecture & verification guidance) paired with [OWASP API Security Top 10](#).

Focus lenses relevant to heavyweight PHP backends:

- Broken object-level authorisation (**BOLA**)
- Authentication fragilities & token replay classes
- Excessive truthful data exposure camouflaged as completeness
- Server-side unsafe request routing classes
- Systemic misconfiguration debt vs component-only blame deflection

Map each lens to illustrative defensive backlog items—not buzzword tagging alone.

Phase — 10-labs-progression-to-pen200

Directory: 10-labs-progression-to-pen200/

Source: 10-labs-progression-to-pen200/01-roadmap-resources-and-policy.md

Authorised labs → PEN-200 preparedness — roadmap & ethos

Purpose: unify prior Linux, networking, web security, Burp/HTTP ergonomics, Windows / AD familiarity, toolchain maturity, and systematic enumeration instincts into **repeatable authorised lab choreography**, conscientious notebooks, troubleshooting maturity—not unstructured “spray boxes” boasting.

Symbolic progression (**platform inventories rename—mirror current vendor catalogues faithfully**):

```
TryHackMe Jr penetration tester groundwork
→ HackTheBox calibrated Easy arcs
→ selected Medium escalation bridges
→ OffSec-aligned Proving Grounds Practice tenor
→ self-audited readiness for formal PEN-200 enrollment
```

OSCP credential attempt remains optional, conditioned on preparedness, affordability, organisational policy adherence, respectful assessment ethics.

Reference surface table (non-exhaustive):

Surface	Guidance
TryHackMe	Jr Penetration Tester corridor + methodological augment rooms
HackTheBox	Progressive Easy then curated Medium machines—respect platform ToS/lawful use
HackTheBox Academy	Structured OSCP orientation segments
OffSec proving grounds labs	Style alignment before committing PEN-200 tuition pacing
PEN-200 course	After disciplined readiness—you schedule per OffSec process

Supporting complementary narrative courses (**verify storefront currency**):

- Udemy arcs often titled variants of “**Hack The Box → OSCP**” spanning multiple instalments (+ optional bonus boxed expansions)—always confirm legitimacy & freshness before purchase **if** desired.

Chosen note toolchain (Obsidian vs CherryTree vs Joplin vs plain Markdown vault) — **privacy under your stewardship**.

Source: 10-labs-progression-to-pen200/02-jr-pen-path-footprinting.md

Unit 02 — Jr penetration tester path — footprint rehearsal

Formalise sanctioned **Junior Pen Tester** syllabus segments stressing **authorised information gathering**.

Pair platform lessons with restrained local **nmap** corroborations on VMs you erected—avoid collateral Wi-Fi probing indiscretion mirroring careless conference behaviour.

Source: `10-labs-progression-to-pen200/03-jr-path-enumeration-workflow-notes.md`

Unit 03 — Enumeration workflow & notebook scaffolding

Operationalise repeatable template sections:

Hostname snapshot • Listener inventory • surfaced service guesses • tentative hypotheses • validation experiment trail • artefacts / artefacts hash references • rollback idea if lab disposable

Maintain discipline **prior** chasing external write-ups—articulate deliberate stuck signals first.

Source: `10-labs-progression-to-pen200/04-web-stack-analysis-hygiene.md`

Unit 04 — Structured web confrontation mindset (Symfony + JWT + OAuth + Burp)

Cross-pollinate HTB-aligned narrative courses (subset you elect) alongside **authorised local Symfony playgrounds**.

Trace narrative:

Plaintext / TLS HTTP surfaces → nuanced header interplay → brittle cookie choreography → JWT lifecycles glimpsed ethically → OAuth exchange cadence critiques → brittle authorisation branching reviews.

Source: `10-labs-progression-to-pen200/05-smb-and-windows-machine-workflow-easy-tier.md`

Unit 05 — SMB & Windows-accented authorised machines (controlled Easy tier climbs)

Iterate Windows/SMB-accented sanctioned machines **inside platform rules**.

Chain inspection:

Controlled port summaries → deliberate SMB hypotheses → restrained LDAP/Kerberos touchpoints aligning earlier AD theory—not exploit-first adrenaline amnesia.

Source: `10-labs-progression-to-pen200/06-independent-attempt-before-hints.md`

Unit 06 — Integrity rule: endeavour before spoilers

Maintain personal rule variants such as:

1. Exhaust reasoned hypotheses & structured note updates **solo** inside authorised lab quotas.
 2. Only afterward consult sanctioned **hint tiers** respecting platform etiquette—not immediate walkthrough ingestion eroding formative struggle.
-

Source: `10-labs-progression-to-pen200/07-medium-difficulty-integration.md`

Unit 07 — Moderate climbs synthesising accrued theory

Proceed to carefully chosen **Medium** difficulty spectrum machines—remain inside explicit platform licence & personal risk tolerance gates.

Iterate workflow loop:

Breadth enumeration → narrowing hypothesis cones → reproducible validations → escalating complexity only alongside explicit notebook maturity checks.

Source: `10-labs-progression-to-pen200/08-offsec-proving-grounds-practice-tone.md`

Unit 08 — Proving Grounds Practice — temperament alignment

Operate within [OffSec practice lab contracts](#) you legitimately subscribe to.

Emphasise **documentation cadence**, time-box stewardship (qualitative—not numeric mandates here), iterative hypothesis resilience echoing PEN-200 lab atmosphere—without circumventing coursework honour codes.

Source: `10-labs-progression-to-pen200/09-udemy-htb-oscp-bridge-mindset.md`

Unit 09 — External HTB → OSCP bridging narrative courses (optional adjuncts)

If you elect paid bridge courses on learning marketplaces (**verify listing titles, reviews, refund policy, legal provenance** before purchase), treat them as **workflow pattern commentary**—not substitutes for lab hours on authorised platforms.

Typical storefront cluster (URLs change—open from course pages only after you validate the syllabus):

Part	Link
Part 1 (workflow intro)	Udemy → HTB to OSCP (part 1)
Part 2	Udemy → HTB to OSCP (part 2)
Bonus / extra boxes	Udemy → HTB to OSCP (bonus)

Use alongside **authorised** THM / HTB / Proving Grounds practice—not as your only reps.

Source: `10-labs-progression-to-pen200/10-pen200-self-audit-gate.md`

Unit 10 — PEN-200 readiness interrogation (no external exam breach talk)

Answer **honestly—without notes on first pass** concept check categories such as:

Comfort narrating Linux troubleshooting paths? Core networking failure triage? JWT/OAuth defensive reviews? LDAP bind & query literacy? Kerberos vs NTLM comparison at high level? SMB share permission reasoning? Structured HTTP review? Burp repeatable manipulations? Controlled `nmap` / `ffuf` / Gobuster usage on owned assets?

If material gaps surface, **loop earlier OSCP+ areas** deliberately before funding formal coursework—integrity over vanity acceleration.

Source: `10-labs-progression-to-pen200/11-pen200-coursework-ethics.md`

Unit 11 — Formal PEN-200 engagement (post-readiness only)

Upon purchasing / licensing [PEN-200](#) per OffSec process:

Complete interactive curriculum materials, challenge labs, supplementary exercises—maintaining **assessment honour code** vigilance (never discuss live restricted exam content / machine-specific spoiling).

Your external certificate attempt remains **optional**.

Source: `10-labs-progression-to-pen200/12-full-path-integration-narrative.md`

Unit 12 — Full-path integration diagram (conceptual)

Articulate end-to-end thematic spine you traversed (customise wording to your actual depth honestly):

```
Linux fluency
→ Networked traffic reasoning
→ Cybersecurity mindset / pre-security orientation
→ Deep web + API + auth security engineering study
→ Instrumented Burp-heavy HTTP review craft
→ Enterprise Windows + AD literacy
→ Consolidated Kali-style tooling repetition
→ Enumeration + relationship mapping methodology
→ Sequential authorised lab ladders
→ PEN-200 formalisation (optional OSCP attempt)
```

Celebrate **method + documentation + ethics**—not machine-count scoreboards alone.
